

Branching Deep Q-Network Enhanced Memetic Algorithm for Skilled-Worker Constrained Flexible Job Shop Scheduling Problem

Chi Zhang, Quanke Pan, *Member, IEEE*, Ling Wang, *Senior Member, IEEE*, Zhonghua Miao, Weimin Li

Abstract—The flexible job shop scheduling problem (FJSP) traditionally emphasizes machine flexibility, while worker heterogeneity is often simplified or neglected. In practical manufacturing systems, worker skill levels affect both processing efficiency and task feasibility, leading to the skilled-worker constrained flexible job shop scheduling problem (SWC-FJSP). This article investigates the SWC-FJSP with multi-level worker skills and skill-dependent processing efficiency, aiming to minimize the makespan. A mixed-integer linear programming model is first formulated to characterize the coupled operation-machine-worker assignment decisions. Then, a branching deep Q-network enhanced memetic algorithm, termed MABDQN, is developed as a learning-assisted evolutionary framework. MABDQN integrates an adaptive hybrid initialization strategy to generate high-quality and diverse populations, a BDQN-driven operator orchestration mechanism to select operator category and execution intensity according to the evolving search state, and a multi-stage adaptive restart strategy guided by a stage adaptive reward function to alleviate premature convergence. Extensive experiments on a newly constructed test set with different instance scales show that MABDQN outperforms several state-of-the-art algorithms in solution quality and robustness. Ablation studies further verify the contribution of the main components and demonstrate the effectiveness of state conditioned operator orchestration for complex SWC-FJSP instances.

Index Terms—Flexible job shop scheduling, Branching deep Q-network, Memetic algorithm, Worker skill level.

I. INTRODUCTION

IN MODERN manufacturing systems, the flexible job shop scheduling problem (FJSP) [1] extends the classical job shop scheduling problem (JSP) by introducing machine selection flexibility, allowing each operation to be processed on alternative machines with varying processing times [2]. However, in practical production environments [3], particularly those involving high precision, specialized expertise, or manual operations, worker heterogeneity plays an equally critical

role. Workers differ in proficiency and skill levels [4], leading to substantial variations in processing efficiency even when executing identical operations on the same machine. Certain complex or critical quality operations further impose minimum skill requirements, restricting feasible assignments to adequately qualified workers [5]. Neglecting such heterogeneity yields schedules misaligned with real-world production constraints and diminishes optimization effectiveness [6].

Building upon the joint consideration of worker heterogeneity and machine flexibility [7], the skilled-worker constrained flexible job shop scheduling problem (SWC-FJSP) models workers and machines as interdependent production resources. In this problem, an operation can only be executed if it is assigned to both a capable machine and a worker whose skill level meets the specific task requirement [5]. This formulation closely reflects the complexity of labor- and equipment-intensive environments such as high-end equipment manufacturing [8] and aerospace production [9]. In these industries, specialized human expertise has a decisive influence on the overall scheduling performance.

Worker skill heterogeneity is not simply a linear factor that scales processing times. Instead, it acts as a strict hard constraint for feasibility. This requirement creates a stringent three-way coupling among operations, machines, and workers. In conventional scheduling models, altering a machine assignment usually preserves the feasibility of the schedule. However, in the SWC-FJSP, assigning a new machine often dictates finding a different worker whose skill level matches the specific demands of that new combination. Consequently, any mismatch instantly leads to an infeasible schedule. This strict qualification mechanism severely fragments the feasible solution space. Navigating this highly restricted space adds immense combinatorial complexity, making the search for optimal schedules extremely challenging. Despite growing research interest, most existing studies still employ simplified worker-skill models [10]. These models, typically binary or discrete, fail to quantitatively capture this strict relationship between skill proficiency and assignment feasibility, limiting their realism in complex scenarios.

In the SWC-FJSP, the computational challenge arises from the interaction between resource feasibility and skill-dependent processing times. Each operation must be assigned to a feasible machine and a qualified worker, and changing either resource may alter the feasible machine-worker combinations, the actual processing time, and subsequent resource conflicts. This coupling fragments the feasible space and makes local schedule modifications highly dependent on the current resource state [11]. In the MILP formulation, it further requires joint operation-machine-worker assignment

This work is supported in part by the National Nature Science Foundation of China 62273221, Program of Shanghai Academic/Technology Research Leader 21XD1401000, and Shanghai Key Laboratory of Power station Automation Technology. (Corresponding authors: Quanke Pan).

Chi Zhang is with the School of Mechatronic Engineering and Automation, Shanghai University, Shanghai 200444, P. R. China (e-mail: quirrell0414@shu.edu.cn).

Quanke Pan is with the School of Mechatronic Engineering and Automation, Shanghai University, Shanghai 200444, P. R. China; and the School of Computer Science, Liaocheng University, Liaocheng 252000, P. R. China (e-mail: panquanke@shu.edu.cn).

Ling Wang is with the Department of Automation, Tsinghua University, Beijing 100084, P. R. China (e-mail: wangling@tsinghua.edu.cn).

Zhonghua Miao is with the School of Mechatronic Engineering and Automation, Shanghai University, Shanghai 200444, P. R. China (e-mail: zhhmiao@shu.edu.cn).

Weimin Li is with the School of Mechatronic Engineering and Automation, Shanghai University, Shanghai 200444, P. R. China (e-mail: wml@shu.edu.cn).

variables and disjunctive sequencing constraints for both machines and workers, causing the model size to increase rapidly with the number of operations and resources. Conventional metaheuristics also face difficulties under such tight coupling, since generic operators may disrupt useful machine-worker matching structures and fixed operator settings cannot respond effectively to changing population states and resource bottlenecks [12]. An adaptive search mechanism is therefore needed to coordinate sequencing and resource assignment under evolving search states.

To overcome these challenges, we develop an enhanced memetic algorithm (MA) integrated with a branching deep Q-network (BDQN), termed MABDQN, which introduces adaptive learning based decision making into the evolutionary search process. The core idea is to leverage the learning capability of BDQN to dynamically orchestrate the search behavior of the MA, achieving intelligent control over resource allocation and operation sequencing. The main contributions of this work are summarized as follows:

- 1) A mixed-integer linear programming (MILP) model is established for the SWC-FJSP with multi-level worker skills. The model characterizes machine-worker flexibility, skill-based feasibility, and skill-dependent processing efficiency, providing a rigorous basis for evaluation and benchmarking.
- 2) An adaptive hybrid initialization (AHI) strategy is developed to generate high-quality and diverse initial populations. It extracts instance features to identify bottlenecks, maps them to nine heuristic rules, and refines rule allocation through proxy evaluation.
- 3) A BDQN-driven operator orchestration mechanism is designed to guide memetic search online. By perceiving evolutionary states and resource bottlenecks, it adaptively selects operator categories and execution intensities to coordinate operation sequencing, machine allocation, and worker assignment.
- 4) A multi-stage adaptive restart strategy is proposed to overcome stagnation in the fragmented feasible space. Guided by stage adaptive rewards, it combines elite retention, composite strong mutation, and population reconstruction to improve global search robustness.

The remainder of this paper is organized as follows. Section II presents the literature review and problem description. Section III formulates the MILP model of SWC-FJSP. Section IV describes the proposed MABDQN algorithm in detail. Section V reports and analyzes the experimental results. Finally, Section VI concludes the paper and outlines future research directions.

II. RELATED WORKS

A. Skilled-worker Constrained Flexible Job Shop Scheduling Problem

The SWC-FJSP (Fig. 1) simultaneously considers machine and worker constraints [13]. It involves determining job sequences, machine allocation, and worker assignment under dual-resource restrictions. Each operation can be processed on multiple alternative machines, and its execution further

requires a qualified worker. Compared with the classical FJSP, the SWC-FJSP introduces stronger coupling among operations, machines, and workers, because a machine assignment may change both the feasible worker set and the actual processing time. As the number of jobs, operations, and resources increases, the solution space grows exponentially, making exact optimization impractical for large-scale cases.

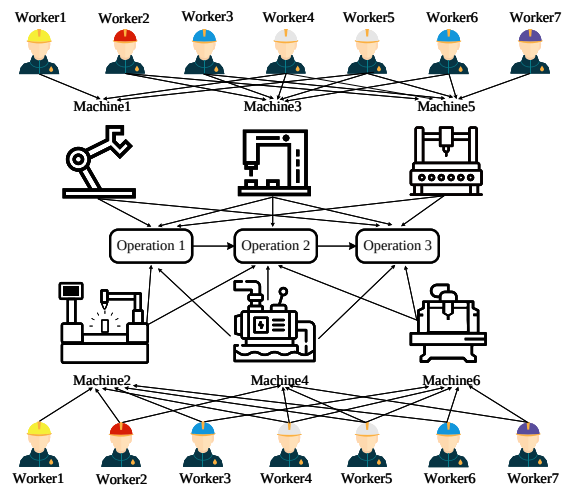


Fig. 1. Schematic of the skilled-worker constrained flexible job shop scheduling problem, where worker skill levels are indicated by safety helmet color: white (level 5), blue (level 4), red (level 3), purple (level 2), and yellow (level 1).

The SWC-FJSP is widely encountered in manufacturing, process industries, and service systems. In automotive production, component machining depends on coordinated scheduling of machines and skilled operators [14]. In chemical industries, scheduling must consider equipment availability, catalyst cycles, and qualified operating teams [15]. In healthcare, surgical scheduling aligns operating rooms, surgeons, anesthesiologists, and nursing staff to ensure efficiency and safety [16]. These scenarios indicate that worker skills affect not only processing efficiency, but also assignment feasibility and schedule robustness.

Although significant progress has been achieved in FJSP research, the SWC-FJSP remains challenging due to dual-resource coupling and the rapid growth of the search space [17]. Li et al. [18] proposed a hybrid genetic algorithm incorporating worker heterogeneity but only considered discrete skill levels. Li et al. [19] adopted an improved NSGA-II for skilled-worker scheduling, though its local search ability remained limited. Li et al. [20] introduced a critical-path based variable neighborhood search, which enhanced exploitation but was sensitive to initial solutions. These studies show the potential of metaheuristics, but also reveal the need to coordinate global exploration and local refinement under complex machine-worker interactions.

B. Memetic Algorithm

MAs combine global exploration with local refinement and have proven effective for NP-hard scheduling problems [13], [21]. By integrating the exploration ability of evolutionary

algorithms with local optimization, MAs can search diverse feasible regions while refining promising solutions, thereby improving solution quality [22].

In scheduling, MAs have achieved notable results. Wei et al. [23] hybridized genetic algorithms with simulated annealing (SA), where SA enabled probabilistic acceptance of inferior solutions to escape local optima. For the SWC-FJSP, Gong et al. [24] developed a knowledge guided MA using genetic algorithms for global search and dynamically selected heuristic rules for local improvement, though its reliance on predefined rules limited adaptability. Zhang et al. [25] developed an adaptive dual-population MA for the FJSP with worker flexibility and outsourcing time windows. Although it improves exploration-exploitation coordination, preserving high-quality structures under strong resource coupling remains challenging.

MAs demonstrate strong potential for the SWC-FJSP, yet their performance depends heavily on how global and local operators are coordinated during the search. In this problem, an operator may affect operation sequencing, machine allocation, and worker assignment simultaneously, so a fixed or manually designed operator schedule may become unsuitable when the population state or resource bottleneck changes. Most existing works still rely on static or problem specific configurations, lacking adaptive mechanisms to adjust operator strategies according to evolving search states. This limitation motivates learning-assisted evolutionary computation (EC), where learning mechanisms are introduced to extract search information and guide evolutionary behavior dynamically.

C. Learning-Assisted Evolutionary Search

Learning-assisted evolutionary search has become an important direction for enhancing the adaptivity and efficiency of EC. Earlier studies [26] on the interaction between EC and machine learning have shown that learning models can help optimization algorithms exploit data, search experience, and problem structures more effectively. Zhan et al. [27] proposed the learning aided evolution for optimization framework, emphasizing that useful experience extracted from evolutionary search can be fed back to improve optimization performance. Jiang et al. [28] further proposed the knowledge learning for EC framework, where evolutionary data are transformed into reusable knowledge for guiding subsequent search behaviors.

Recent advances have expanded this idea from general hybridization to more concrete learning roles in evolutionary algorithms. Learning models have been used as evaluators for expensive fitness approximation [29], generators for producing promising solutions [30], transfer modules for reusing knowledge across tasks [31], and controllers for adapting parameters, operators, or search directions. These developments indicate that learning-assisted EC is not limited to replacing one component of an algorithm; its central issue is how learned information is embedded into the evolutionary process and how it changes the behavior of the search.

At the algorithm execution level, adaptive operator selection (AOS) provides a representative way to adjust search behavior online. UCB based selection [32], adaptive pursuit [33], and probability matching [34] adjust operator usage according to

historical performance and credit assignment. However, such stateless mechanisms mainly rely on accumulated feedback and do not explicitly perceive the current optimization state. This limitation is critical for the SWC-FJSP, where operator effectiveness changes with population diversity, convergence status, and machine-worker bottlenecks. Therefore, learning-assisted search in this problem should move beyond historical operator preference and use real-time population and resource information to control the search behavior during evolution.

D. Branching Deep Q-Network

BDQN provides an architectural solution to the structured control requirement described above. In standard DQN, a multidimensional decision is usually flattened into a single action set, which requires enumerating all action combinations and may reduce learning efficiency in combinatorial search. BDQN avoids this issue by using a shared feature extractor and multiple parallel branches, each estimating Q-values for one action dimension [35]. As shown in Fig. 2, the branch outputs are aggregated to form the final action, enabling coordinated decisions without exponential action expansion.

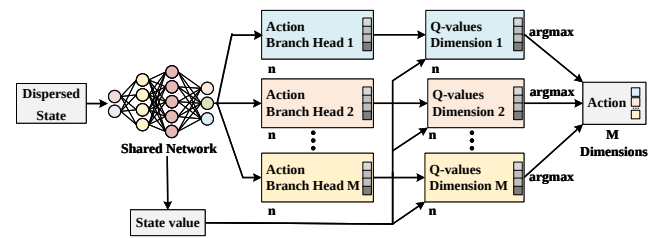


Fig. 2. Branching Deep Q-Network Architecture.

This structure is suitable for scheduling scenarios where several coupled decisions must be made simultaneously. Liu et al. [36] applied BDQN to shop scheduling with graph based state representation, and Zheng et al. [37] further demonstrated its effectiveness in dynamic environments. For the SWC-FJSP, BDQN is promising because it provides a compact way to connect state aware learning with evolutionary search under changing population and resource conditions. Its effective use still requires problem specific state representation and feedback design, so that the learned policy can guide the evolution of the population and be updated by the search outcomes.

E. Research Gaps

The SWC-FJSP continues to draw research attention due to its modeling complexity and practical significance. Although various heuristic and learning-assisted approaches have improved scheduling efficiency, several challenges persist:

- 1) Existing models often oversimplify worker heterogeneity and fail to quantitatively represent how skill proficiency affects processing efficiency and assignment feasibility.
- 2) Traditional metaheuristics have shown potential for the SWC-FJSP, but most still rely on static or manually designed search configurations, limiting their adaptation to changing population states and machine-worker bottlenecks.

- 3) learning-assisted EC has advanced in surrogate modeling, knowledge transfer, parameter adaptation, and operator control, but state and resource aware learning for the SWC-FJSP remains underexplored.

III. PROBLEM DESCRIPTION AND FORMULATION

A. Problem Description

This article addresses a scheduling problem in a complex manufacturing environment, namely the SWC-FJSP, with emphasis on worker skill differences and their impact on processing efficiency. The system consists of n jobs ($J_1, J_2, \dots, J_n$), each comprising one or more operations in a predefined sequence. The h -th operation of job j is denoted O_{jh} . The shop contains m machines ($M_1, M_2, \dots, M_m$) and w workers ($W_1, W_2, \dots, W_w$). Each operation O_{jh} can be processed on one of several candidate machines that meet its technological requirements, and execution requires both a machine and a worker.

Workers differ in skill levels SK , with a minimum of 1, and each machine–operation pair has a minimum required skill MS_{jh} . Only workers meeting or exceeding this requirement can perform the operation. Worker skill directly affects processing efficiency: for each level above the minimum, the operation time is reduced by 10%. The scheduling objective is to assign each operation to an appropriate machine and qualified worker, determine start times for all operations, and minimize the makespan C_{max} , while respecting precedence constraints and the uniqueness of machine and worker assignments.

TABLE I
ILLUSTRATIVE SWC-FJSP INSTANCE WITH BASIC PROCESSING TIMES
AND SKILL CONSTRAINTS

Job	Op.	MS_{jh}	SK_1	SK_2	Worker Set	M_1		M_2		M_3	
						W_1	W_2	W_1	W_2	W_1	W_2
J_1	O_{11}	1	2	1	W_1, W_2	—	—	198	144	59	—
	O_{12}	1	2	1	W_1, W_2	—	—	84	91	—	—
	O_{13}	2	2	1	W_1	157	134	—	—	157	—
J_2	O_{21}	2	2	1	W_1	150	62	142	—	—	54
	O_{22}	1	2	1	W_1, W_2	218	159	71	—	148	105
J_3	O_{31}	1	2	1	W_1, W_2	198	—	—	187	—	232
	O_{32}	2	2	1	W_1	115	—	108	—	—	—
	O_{33}	1	2	1	W_1, W_2	—	—	55	223	—	—

The SWC-FJSP can be divided into three subproblems: (1) selecting the most suitable machine for each operation; (2) assigning a qualified worker meeting the required skill level; and (3) sequencing operations on each machine under worker constraints to minimize makespan. To illustrate, Table I presents a compact example with eight operations from three jobs (3/2/3), three machines, and two workers. Here, SK_1 and SK_2 denote the skill levels of W_1 and W_2 , respectively, and Worker Set denotes the workers satisfying the skill requirement. The processing-time columns list basic processing times before skill filtering; “—” denotes an unavailable machine-worker processing time. Thus, a feasible assignment must have an available processing time and a worker contained in Worker Set. Each operation must satisfy a minimum skill requirement. For instance, O_{21} requires skill level 2, so only W_1 satisfies the skill requirement. The actual processing time

on M_1 is calculated as the basic processing time multiplied by a skill-based coefficient, e.g., $150 \times [1 - (2 - 1) \times 0.1] = 135$, as defined in Eq. (1).

$$\tilde{T}_{jh mw} = T_{jh mw} \times [1 - \eta \cdot (\max(0, SK_w - 1))] \quad (1)$$

where $T_{jh mw}$ is the basic processing time, $\eta = 0.1$ is the efficiency factor, and $\tilde{T}_{jh mw}$ is the actual processing time adjusted by SK_w . This quantification allows the model to utilize pre-calculated time matrices and eligible worker sets, avoiding redundant skill matching and streamlining logical complexity. Although modeled linearly for tractability, the efficiency reduction forces a trade-off between assigning an operation immediately to a slower worker versus waiting for a highly skilled one, increasing combinatorial complexity.

B. MILP Model of SWC-FJSP

Objective:

$$\text{Min } C_{max} \quad (2)$$

Subject to:

$$\sum_{m=1}^M \sum_{w=1}^W x_{jh mw} = 1, \forall j, h \quad (3)$$

$$x_{jh mw} \leq Q_{jh mw}, \forall j, h, m, w \quad (4)$$

$$c_{jh} = s_{jh} + \sum_{m=1}^M \sum_{w=1}^W (x_{jh mw} \cdot \tilde{T}_{jh mw}), \forall j, h \quad (5)$$

$$c_{jh} \leq s_{j(h+1)}, \forall j, h = 1, 2, 3, \dots, h_j - 1 \quad (6)$$

$$s_{kl} \geq c_{jh} - L \cdot y_{jh klm} - L \cdot (2 - X_{jhm} - X_{klm}), \quad (7)$$

$$\forall (j, h) \neq (k, l), m$$

$$s_{jh} \geq c_{kl} - L \cdot (1 - y_{jh klm}) - L \cdot (2 - X_{jhm} - X_{klm}), \quad (8)$$

$$\forall (j, h) \neq (k, l), m$$

$$s_{kl} \geq c_{jh} - L \cdot z_{jh klw} - L \cdot (2 - X_{jhw} - X_{klw}), \quad (9)$$

$$\forall (j, h) \neq (k, l), w$$

$$s_{jh} \geq c_{kl} - L \cdot (1 - z_{jh klw}) - L \cdot (2 - X_{jhw} - X_{klw}), \quad (10)$$

$$\forall (j, h) \neq (k, l), w$$

$$y_{jh klm} \leq X_{jhm}, \forall (j, h) \neq (k, l), m \quad (11)$$

$$y_{jh klm} \leq X_{klm}, \forall (j, h) \neq (k, l), m \quad (12)$$

$$z_{jh klw} \leq X_{jhw}, \forall (j, h) \neq (k, l), w \quad (13)$$

$$z_{jh klw} \leq X_{klw}, \forall (j, h) \neq (k, l), w \quad (14)$$

$$c_{jh_j} \leq C_{max}, \forall j \quad (15)$$

$$s_{jh} \geq 0, \forall j, h \quad (16)$$

$$c_{jh} \geq 0, \forall j, h \quad (17)$$

$$x_{jh mw} \in \{0, 1\}, \forall j, h, m, w \quad (18)$$

$$y_{jh klm} \in \{0, 1\}, \forall (j, h) \neq (k, l), m \quad (19)$$

$$z_{jh klw} \in \{0, 1\}, \forall (j, h) \neq (k, l), w \quad (20)$$

$$C_{max} \geq 0 \quad (21)$$

Detailed symbol definitions and the MILP model description are provided in Section I of the Supplementary Materials.

C. Validation of MILP Model

The proposed SWC-FJSP MILP model was verified using CPLEX 22.1.0 on ten small-scale instances from S01 to S10. As summarized in Table II, CPLEX obtains exact optimal solutions for all instances, confirming the correctness of the formulation on tractable cases. The MILP model also provides exact benchmarks for validating the decoding procedure, agent actions, reward design, and search quality of MABDQN. These results show that the MILP model is useful for small-instance validation, while the rapid growth in solution time motivates an adaptive metaheuristic solver for larger SWC-FJSP instances.

TABLE II
COMPUTATIONAL RESULTS OF THE CPLEX.

Ins.	n	m	w	$max\ h_j$	T_0	Time/s	C_{max}
S01	3	3	2	3	7	1.62	88
S02	4	3	2	3	8	1.73	82
S03	5	3	2	3	9	8.86	98
S04	4	4	3	4	11	12.46	69
S05	4	5	3	4	12	36.79	66
S06	4	5	4	4	13	111.52	70
S07	3	4	3	5	14	126.8	87
S08	4	4	5	5	15	379.17	75
S09	5	5	4	4	16	404.29	83
S10	6	4	3	4	17	609.64	115

IV. THE PROPOSED ALGORITHM

A. Framework of MABDQN

To address the complex SWC-FJSP, an enhanced MA integrated with a BDQN is proposed. The algorithm tightly couples MA, featuring an adaptive initialization strategy and dynamic restart mechanism, with BDQN. In this framework, the evolutionary search is responsible for generating and improving candidate schedules, while the BDQN agent learns how to guide this search according to the current population state. The RL agent orchestrates a set of operators crossover (CR), mutation (MT), and neighborhood search (NS) based on the MA's evolving state, enabling effective exploration and intensive exploitation of the solution space. After the selected operators generate offspring, their search effect is fed back to the agent through the reward function, so that the operator control policy can be updated along with the evolution of the population.

Algorithm 1 summarizes the overall framework of MABDQN. The algorithm first generates the initial population P by adaptive hybrid initialization using the instance data I and population size PS , evaluates P , and determines the initial global best solution g^* ; meanwhile, the BDQN agent A , the replay buffer $\mathcal{D}$, and the stagnation counter c are initialized before the evolutionary process starts (lines 1–3). In each generation, the current population is converted into the state vector s , and the BDQN agent selects the action $a = (a_{cat}, a_{int})$, where a_{cat} denotes the selected operator category and a_{int} denotes the corresponding intensity level. Then, the parent set M is selected, the offspring population Q is generated under the selected action, and the Stage adaptive reward r is computed according to the current evolutionary phase ϕ_g . After elitist replacement, the current best solution

Algorithm 1 MABDQN

Input: Instance data I , population size PS , maximum generation G_{max} , stagnation threshold T_s
Output: Global best solution g^*

- 1: Generate initial population P by adaptive hybrid initialization using I and PS
- 2: Evaluate P and set $g^* \leftarrow \text{best}(P)$
- 3: Initialize agent A , replay buffer $\mathcal{D}$, and stagnation counter c
- 4: **for** $g = 1$ to G_{max} **do**
- 5: Extract state vector s from population P
- 6: Select action $a = (a_{cat}, a_{int})$ by agent A
- 7: Select parent set M and generate offspring population Q according to action a
- 8: Evaluate Q and compute reward r with phase $\phi_g \leftarrow g/G_{max}$
- 9: Update P by elitist replacement using Q and g^* ; set $b \leftarrow \text{best}(P)$
- 10: **if** $C_{max}(b) < C_{max}(g^*)$ **then**
- 11: $g^* \leftarrow b$, $c \leftarrow 0$
- 12: **else**
- 13: $c \leftarrow c + 1$
- 14: **end if**
- 15: **if** $c \geq T_s$ **then**
- 16: Reconstruct population P by adaptive restart using g^*
- 17: Evaluate P and reset $c \leftarrow 0$
- 18: **end if**
- 19: Set next state $s' \leftarrow \text{State}(P)$
- 20: Store transition (s, a, r, s') in $\mathcal{D}$ and update agent A
- 21: **end for**
- 22: **return** g^*

b is identified from the updated population (lines 4–9). If b improves the makespan of the global best solution, g^* is updated and the stagnation counter c is reset; otherwise, c is increased to record one generation without improvement (lines 10–14). When c reaches the threshold T_s , the adaptive restart mechanism reconstructs the population while retaining the global best solution, and the counter is reset (lines 15–18). Finally, the next state s' is extracted, the transition (s, a, r, s') is stored in the replay buffer $\mathcal{D}$, and the BDQN agent is updated using mini-batch samples. The process continues until G_{max} is reached, and the algorithm returns the global best solution g^* (lines 19–22). Through this iterative process, the population state, operator action, offspring evaluation, and policy update are linked in each generation, which makes the BDQN agent an online controller of the memetic search rather than a detached post processing module.

B. Chromosome Encoding and Decoding

1) *Encoding scheme:* A three segment integer encoding is proposed to represent a complete SWC-FJSP schedule during evolution (Fig. 3). Each individual comprises operation sequencing (OS), machine selection (MS), and worker scheduling (WS). The OS segment uses job based encoding, where the sequence of job numbers determines the operation order (e.g., $OS = [2\ 1\ 3\ 2\ 3\ 1\ 1\ 3]$ corresponds to $[O_{21}, O_{11}, O_{31}, O_{22}, O_{32}, O_{12}, O_{13}, O_{33}]$, Fig. 3(a)). In contrast, MS and WS follow the fixed operation order (Fig. 3(b)), with each gene indicating the index of a machine or worker in the candidate set rather than its actual ID, ensuring feasibility after CR, MT, and NS. Collectively, the OS, MS, and WS segments define a complete and feasible schedule.

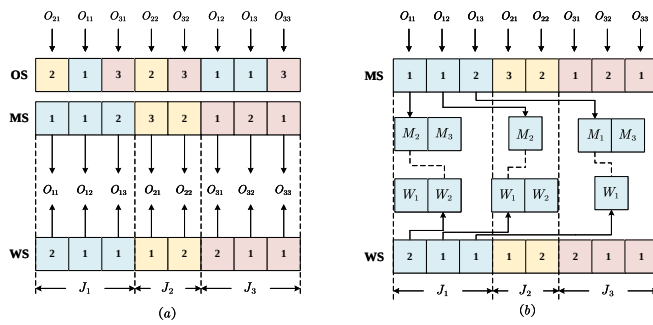


Fig. 3. Structure of the chromosome encoding for operation, machine, and worker assignments.

2) *Decoding scheme*: Decoding converts the three segment chromosome into an executable schedule. The algorithm employs an active scheduling strategy, with makespan as the fitness criterion. During initialization, static resource time mapping assigns each operation to a machine and worker based on the MS and WS segments, determining processing time from the indirect indices. Subsequently, a dynamic simulation proceeds according to the global priority defined by the OS segment. For each operation, the earliest start time (EST) is computed as the maximum of three constraints.

- **Job Precedence**: completion time of the immediately preceding operation of the same job;
- **Machine Availability**: completion time of the last task on the assigned machine;
- **Worker Availability**: completion time of the last task performed by the assigned worker.

After determining EST, the operation's completion time is updated, and the states of the associated job, machine, and worker are refreshed. A global variable continuously tracks the maximum completion time, which upon scheduling all operations, defines the makespan and serves as both the chromosome's fitness and the reward signal for learning.

C. Adaptive Hybrid Initialization

The quality and diversity of the initial population have a profound impact on the performance and convergence efficiency of evolutionary algorithms. Random initialization often leads to aimless search and slow convergence, while relying on a single heuristic tends to cause premature convergence. To generate high-quality, instance specific populations that balance quality and diversity, Algorithm 2 adopts an AHI strategy. The AHI analyzes problem specific characteristics and adaptively orchestrates multiple heuristic rules, providing a tailored and effective starting population for subsequent evolution.

The AHI consists of four stages, each guided by data-driven adaptation rather than fixed heuristics.

- **Stage 1 (Line 1): Feature extraction**. The algorithm quantifies the structural characteristics of the given SWC-FJSP instance, including (1) *machine load imbalance*, measuring the variation in potential total processing time across machines; (2) *job length imbalance*, representing the heterogeneity of total minimum processing times across jobs; (3) *average machine flexibility*, describing

Algorithm 2 Adaptive Hybrid Initialization (AHI)

Input: Population size N , Instance data I

Output: Initial population P

```

1: Extract features  $F \leftarrow \text{Feature}(I)$ 
2: Initialize rule weights  $W \leftarrow \text{InitWeight}(F)$ 
3: if proxy evaluation required then
4:   Refine weights  $W \leftarrow \text{Refine}(I)$ 
5: end if
6: Allocate individuals  $R \leftarrow \text{Allocate}(W, N)$ 
7:  $P \leftarrow \emptyset$ 
8: for all  $r \in R$  do
9:    $n \leftarrow R[r]$ 
10:   $Q \leftarrow \text{Generate}(r, n)$ 
11:   $P \leftarrow P \cup Q$ 
12: end for
13: return  $P$ 

```

the average number of eligible machines per operation; and (4) *average route flexibility*, reflecting the average number of feasible machine-worker combinations per operation. These indicators provide an objective basis for subsequent rule selection and weight assignment, enabling the algorithm to perceive how skill constraints compress the feasible search space.

- **Stage 2 (Line 2): Heuristic pool initialization**. A set of nine complementary heuristic rules is prepared, including *shortest-job-first*, *earliest-finish-time*, *shortest-operation-time*, *minimal-load-first*, *idle-machine-first*, *critical-path-first*, *bottleneck-machine-first*, *rotation-based assignment*, and *random generation*. The algorithm initializes the weights of these rules according to the extracted features. For example, when machine load imbalance is high, the weight of the *bottleneck-machine-first* rule is automatically increased, as it better mitigates resource bottlenecks. By coordinating machine and worker selection, these rules effectively handle the dual-resource coupling inherent in the problem.
- **Stage 3 (Lines 3–5): Proxy based refinement**. Static heuristics may not fully capture the characteristics of specific instances. A lightweight empirical evaluation is therefore introduced. The algorithm generates a small number of candidate solutions using each rule and rapidly evaluates them through a relaxed proxy function. The rule weights are refined according to relative performance, ensuring that the final weight distribution reflects empirical evidence rather than prior estimation. This feedback loop helps identify hidden resource conflicts and skill matching traps that static analysis might overlook.
- **Stage 4 (Lines 6–13): Population construction**. Based on the refined weights, individuals are allocated to each heuristic in proportion to their scores. To maintain exploration capability, diversity-oriented rules such as rotation based and random generation are guaranteed a minimum allocation. The remaining slots are distributed among high weight rules to produce quality seeds. This approach ensures the population covers diverse feasible regions in the fragmented space to prevent early stagnation.

The AHI dynamically balances exploitation and exploration during population generation. By leveraging structural analysis and empirical refinement, it constructs an initial population

that is both diverse and problem aware, providing a well distributed and reliable foundation for the subsequent evolutionary process.

D. Branching Deep Q-Network Assisted Memetic Algorithm Evolution

After constructing a high-quality initial population with the AHI strategy, the MABDQN enters the RL guided evolutionary phase, aiming to adaptively optimize the population until satisfactory solutions are obtained.

1) *State space*: At the start of each generation, the algorithm evaluates the global search state and selects parent individuals, forming the basis for subsequent operations. The adaptive evaluation provides a quantitative description of the search environment for the RL agent. To make effective decisions, the BDQN agent must perceive this environment rather than blindly applying operators.

The seven discretized features are selected to provide a comprehensive representation of both the evolutionary search dynamics and the problem specific resource status. Indicators (1)–(5) describe the global evolutionary state: (1) *population diversity*, (2) *average fitness trend*, (3) *best fitness improvement rate*, (4) *convergence phase*, and (5) *stagnation counter*. These are chosen because they collectively characterize the exploration-exploitation balance and the degree of stagnation, which is critical for operator orchestration in metaheuristics. Indicators (6)–(7), namely (6) *average machine utilization* and (7) *average worker utilization*, reflect the efficiency and balance of resource use. These are specifically tailored to the SWC-FJSP to enable the agent to identify resource bottlenecks and select appropriate resource reallocation operators.

Raw values are discretized into bins to construct a compact state vector, which serves as BDQN input for all RL decisions in that generation. These features allow the agent to observe how the population is evolving and whether the current schedule is affected by machine or worker bottlenecks. Therefore, the learning module receives search context information from the evolutionary process before making operator decisions. Detailed mathematical definitions for the state features and the exact discretization formulas are provided in Section II of the Supplementary Materials.

2) *Two slot multi-action space*: Conventional multi-operator frameworks often adopt fixed CR/MT probabilities or predetermined operator schedules. Such static configurations are inadequate for the inherently dynamic and nonlinear nature of evolutionary search. In the early stages, strong CR is required to exploit population diversity, whereas in later stages disruptive mutations and tailored neighborhood searches are essential to escape local optima. As a result, static settings frequently lead to inefficiency or premature convergence.

To address this limitation, a two slot multi-action space (TMS) driven by BDQN is proposed. Each decision is decomposed into two sub-decisions:

- the first slot selects an operator category (CR, MT, or NS);
- the second slot specifies the execution variant and its intensity level $a_{int} \in \{0, 1, 2\}$.

This hierarchical formulation enables adaptive operator scheduling while avoiding the explosion of action space dimensionality. It also gives the agent two levels of control over the evolutionary search. The category decision determines whether the population is modified through recombination, perturbation, or local refinement, while the intensity decision determines how strongly the selected operator changes the current solutions. The complete operator pool is summarized in Table III.

TABLE III
TWO SLOT MULTI-ACTION SPACE

Category	Strategy	Main operation	Intensity control
CR	OSJC	Job-set OS crossover between two parents	Exchanged job proportion controls disruption degree
	MSS	Job-level MS/WS block exchange between parents	Number of exchanged jobs balances inheritance and exploration
	WSM	Multi-point WS swap for worker allocation	Number of swap points controls worker exploration
MT	OSS	Multi-position OS swap for sequence perturbation	Number of swap pairs controls mutation strength
	MSR	Destroy-and-repair MS/WS for resource reconstruction	Disrupted operation ratio controls repair scale
	WSC	Selected-worker reassignment under efficiency trade-off	Greedy-to-random bias balances quality and diversity
NS	CPPO	Critical-path OS refinement for operation order	Swap, move, or insertion chosen by intensity
	CPRO	Critical-path resource reassignment	Machine, worker, or both are reallocated
	ASAO	Large OS restructuring for diversification	Reversal, relocation, or LNS repair
	GDO	Non-critical-path perturbation	MW reassignment or OS move
	BNSO	Bottleneck-machine task migration	Migration scale controls redistribution
	NO	No operation keeps current solution	No intensity effect

For CR operators (Fig. 4), three strategies are considered: *OS job-centered (OSJC)*, which recombines job clusters; *MS strategy (MSS)*, which exchanges machine and worker assignments at the job level; and *WS multi-point (WSM)*, which swaps worker assignments at multiple positions. The selected strategy is further controlled by the intensity level, determining how disruptive the recombination is.

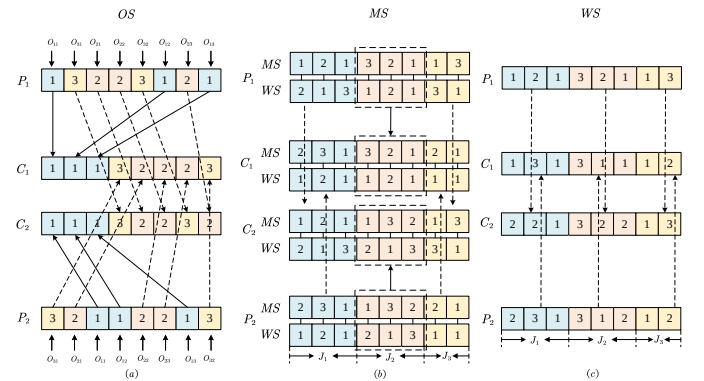


Fig. 4. Illustration of crossover strategies: (a) operation-based, (b) machine-based, and (c) worker-based.

For MT operators (Fig. 5), three schemes are employed: *OS multi-point swap (OSS)*, which perturbs operation sequences;

MS destroy-and-rebuild (MSR), which disrupts and reconstructs resource assignments; and *WS collaborative (WSC)*, which reassigns workers under different efficiency diversity trade-offs. The parameter a_{int} modulates mutation strength, from local adjustments to strong reconstruction.

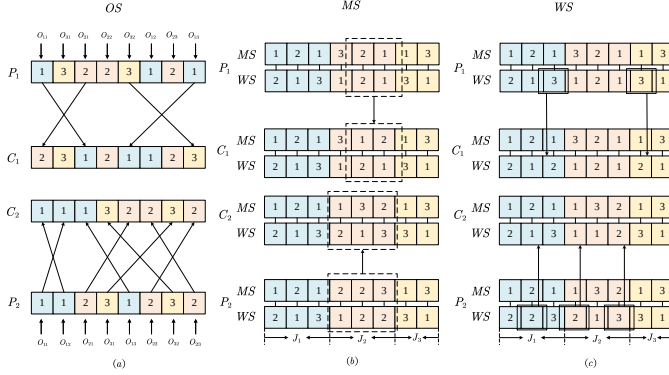


Fig. 5. Illustration of mutation strategies: (a) operation sequence, (b) machine selection, and (c) worker scheduling.

For NS operators, six strategies are available: *critical-path OS point operation (CPPO)*, *critical-path resource operation (CPRO)*, *advanced OS structure adjustment operations (ASAO)*, *general MW disturbance operation (GDO)*, *bottleneck adaptive neighborhood search operation (BNSO)*, and *no operation (NO)*. Each of these strategies incorporates different intensities, e.g., CPPO adjusts the position of a critical-path operation via swap, move, or forward insertion depending on the chosen level.

By coupling the hierarchical TMS with the BDQN policy, the framework achieves dynamic, intensity aware adaptation of operator behavior across different search stages, thereby improving robustness and efficiency in solving complex SWC-FJSP instances. Thus, CR, MT, and NS are not applied according to fixed probabilities; their activation and strength are adjusted according to the learned policy and the current search state.

3) *Network architecture*: A BDQN (Fig. 6) is employed to approximate $Q(s, a)$, mapping the seven-dimensional state space to TMS. The state vector s is first encoded by two shared hidden layers (128 ReLU units each) into a latent representation $h(s)$, which is fed into two parallel branches.

The category branch consists of a 64-neuron ReLU layer and an 8-unit linear output, producing $Q(s, a_{cat})$ for operator categories. The intensity branch, with the same structure but a 3-unit output, estimates $Q(s, a_{int})$ for intensity levels. The final action is composed as Eq. (22).

$$a = (\arg \max_{a_{cat}} Q(s, a_{cat}), \arg \max_{a_{int}} Q(s, a_{int})) \quad (22)$$

This branched design decomposes the combinatorial action space into category and intensity subspaces, preventing dimensionality explosion while enabling adaptive adjustment of perturbation strength. To ensure valid coordination, a conditional mapping logic is applied during decoding. For operators with predefined variants, a_{int} determines the specific intensity. whereas for categories where such levels are not applicable,

the intensity signal is simply ignored to maintain execution stability.

4) *Stage adaptive reward function*: The reward function R provides the feedback by which the BDQN agent evaluates the selected operator. It links the evolutionary outcome of an operator action with the subsequent policy update. In dynamic evolutionary search, a direct single step fitness change may be insufficient, because the same operator can have different effects under different search phases and feasibility states. Therefore, a stage adaptive reward is adopted, where the feedback weights vary across the *Early*, *Mid*, and *Late* phases to match the search emphasis of each stage.

The reward is designed to reflect the contribution of the selected operator to the search process. In the SWC-FJSP, an operator may improve the makespan, deteriorate the current solution, recover feasibility, or produce no effective change. These outcomes provide different learning signals and should be distinguished. Given the offspring makespan C_c and the parental average makespan C_p , the reward R is defined in Eq. (23).

$$R(\phi) = \begin{cases} k_+, & C_p = \infty, C_c < \infty, \\ w_+(\phi)[k_1 + k_2 \tanh(\alpha_+ \Delta)], & C_p < \infty, C_c < C_p, \\ w_-(\phi)[1 + \tanh(\alpha_- |\Delta|)], & C_p < \infty, C_c > C_p, \\ 0, & \text{otherwise.} \end{cases} \quad (23)$$

where $\Delta = (C_p - C_c)/C_p$ denotes the relative makespan change for finite C_p . The fixed reward k_+ is assigned when an infeasible parent produces a feasible offspring, encouraging the search to enter feasible regions. The improvement branch provides positive feedback according to the relative reduction in makespan, whereas the deterioration branch uses $w_-(\phi)$ as a penalty weight to discourage destructive moves. Neutral cases are assigned zero reward to avoid unnecessary updates.

The nonlinear term $\tanh(\cdot)$ is introduced to shape the feedback in a stable manner. For small changes, $\tanh(\cdot)$ behaves almost linearly, allowing the agent to distinguish marginal improvements or deteriorations. For large changes, it gradually saturates, preventing extreme reward values from dominating the Q-value update. In addition, the relative change Δ reduces the scale difference among instances with different makespan ranges. The sensitivity parameters α_+ and α_- control how quickly the reward and penalty respond to solution changes, while k_1 and k_2 ensure that even small improvements can generate positive learning signals. Together with the phase-dependent weights $w_+(\phi)$ and $w_-(\phi)$, the reward function provides feedback that is sensitive to search events, robust to scale variation, and consistent with the changing emphasis of different evolutionary stages.

5) *Learning mechanisms*: Each transition τ_t stores the state, action, reward, and next state generated by one BDQN guided evolutionary step, where $\tau_t = (s_t, a_t, r_t, s_{t+1})$. The MAB-DQN agent employs an off-policy, value based RL framework. To ensure stability in the multi operator environment, it integrates experience replay, target network separation, and double DQN within the BDQN structure.

At each iteration, an experience tuple (s_t, a_t, r_t, s_{t+1}) is stored in a replay buffer. Random mini-batch sampling mit-

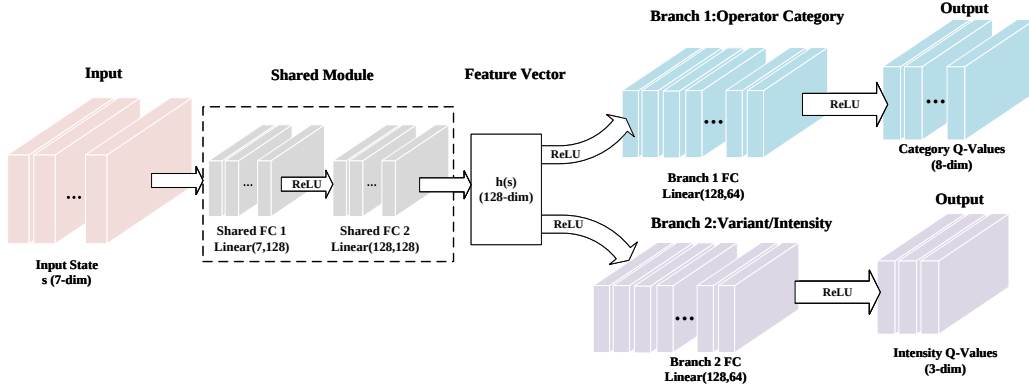


Fig. 6. Architecture of the branching deep Q-network. It gates temporal correlations and enhances data efficiency. Learning updates construct an unbiased target y_t using a separate target network $Q_{\text{target}}(s, a; \theta')$, which shares architecture with but updates more slowly than the main network $Q_{\text{main}}(s, a; \theta)$. For a next state s_{t+1} , the main network independently selects optimal sub-actions per branch (Eq. (24)).

$$a_{t+1,i}^* = \underset{a_i'}{\operatorname{argmax}} Q_{\text{main}}(s_{t+1}, a_i'; \theta) \quad (24)$$

The target network then evaluates these sub-actions, and their Q-values are aggregated to approximate the next state value (Eq. (25)).

$$Q_{\text{next}} = \frac{1}{|\mathcal{A}|} \sum_{i=1}^{|\mathcal{A}|} Q_{\text{target}}(s_{t+1}, a_{t+1,i}^*; \theta') \quad (25)$$

This decoupling of selection and evaluation follows the Double DQN principle, alleviating overestimation. The target is defined via the Eq. (26).

$$y_t = r_t + \gamma Q_{\text{next}} \quad (26)$$

Main network parameters θ are optimized by minimizing branch wise mean squared error against y_t (Eq. (27)).

$$\mathcal{L}(\theta) = \sum_{i=1}^{|\mathcal{A}|} \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim D} [(y_t - Q_{\text{main}}(s_t, a_{t,i}; \theta))^2] \quad (27)$$

Optimization uses Adam with gradient clipping, and θ' is periodically synchronized from θ . This framework enables stable and efficient learning of adaptive operator scheduling policies in complex environments.

E. Adaptive Restart Policy

When search stagnates in late evolution due to deep local optima, MABDQN activates a multi-stage adaptive restart mechanism. The population is rebuilt on a large-scale to restore diversity while preserving high-quality solutions. This process is essential for overcoming skill matching traps inherent in the fragmented search space.

1) *Elite retention*: During disruptive reconstruction, an elitism policy preserves the best individuals as a recovery baseline. A small set of top ranked solutions is retained, with a safeguard ensuring that the global best is always included, replacing the weakest elite if necessary. These elites seed the new population, providing a robust foundation for subsequent exploration across different feasible regions.

2) *Composite strong mutational perturbation*: To avoid renewed stagnation near existing optima, a subset of elites undergoes composite strong mutation (Algorithm 3). This mechanism is designed to break rigid resource task couplings, enabling the population to jump across disconnected feasible regions and ensuring global search robustness. The repaired offspring are incorporated into the rebuilt population, ensuring both the inheritance of elite traits and the expansion of diversity for continued evolution.

Algorithm 3 Composite Strong Mutation

Input: Elite chromosome C_e
Output: Perturbed chromosome C_p
1: $C_t \leftarrow C_e$
2: Disrupt C_t by DestroyRebuild()
3: Rearrange C_t via MultiSwap()
4: Refine C_t through Collaborative()
5: Repair $C_p \leftarrow \text{Repair}(C_t)$
6: **return** C_p

3) *Population restart*: After establishing a high-quality foundation, the remaining slots are filled using a dynamic hybrid strategy (Algorithm 4). It integrates: (i) heuristic rules to generate well structured individuals (line 2), (ii) strong mutation on non-elites (line 3), and (iii) random generation for maximum diversity (line 4). The proportions are dynamically adjusted based on the restart count, more restarts increase the share of strong mutation and randomness while reducing heuristic reliance, thereby enhancing global exploration.

Algorithm 4 Population Restart

Input: Stagnated population P_{old} , Fill size N_f , Restart counter N_r
Output: Reconstructed subpopulation P_f
1: Allocate counts $(n_h, n_m, n_r) \leftarrow \text{Allocate}(N_f, N_r)$
2: Generate heuristic part $I_h \leftarrow \text{HeurGen}(n_h)$
3: Generate mutated part $I_m \leftarrow \text{Mutate}(P_{\text{old}}, n_m)$
4: Generate random part $I_r \leftarrow \text{RandGen}(n_r)$
5: Combine $P_f \leftarrow I_h \cup I_m \cup I_r$
6: **return** P_f

The restart policy also supports the learning-guided search by reshaping the population when long stagnation occurs. The reconstructed population provides new search states for subsequent BDQN-guided operator orchestration, while the retained elites preserve the best search experience obtained so far.

F. Algorithm Complexity Analysis

The computational complexity of MABDQN is primarily determined by initialization and the evolutionary loop. In each generation, the main costs arise from state evaluation, offspring generation, and population assessment. State evaluation, which computes features such as diversity, requires $O(PS \cdot N)$, where PS is the population size and N is the number of operations. Parent selection is $O(PS)$. Offspring generation dominates the evolutionary loop, the evolutionary operators range from $O(N)$ to $O(N^2)$, giving an upper bound of $O(PS \cdot N^2)$ per generation. Population evaluation is $O(PS \cdot N)$. The additional online RL training cost mainly comes from transition storage, mini-batch BDQN updates, and periodic target network synchronization. Since the network structure and mini-batch size are fixed, this cost grows much more slowly than offspring generation and schedule evaluation as the instance scale increases. Adaptive initialization and restart procedures are $O(PS \cdot N)$. Consequently, the overall complexity is governed by offspring generation, and over G generations the complexity is $O(G \cdot PS \cdot N^2)$.

V. EXPERIMENTS AND ANALYSIS

A. Experimental Instances

We construct 30 test instances across three scales as no standard benchmark exists for the SWC-FJSP. Small-scale (DMFJS01 to DMFJS10) and medium-scale (DMK01 to DMK10) sets are adapted from Fattahi [38] and Brandimarte [39] benchmarks. The large-scale set (DMW01 to DMW10) is newly generated to evaluate scalability, covering 60 to 200 jobs, 10 to 30 machines, and 8 to 25 workers.

All instances are extended with worker parameters to fit the SWC-FJSP model. For reproducibility, each job includes 1 to 4 operations. Each operation is assigned to 1 to 5 candidate machines, and each machine is assigned 1 to 4 qualified workers. Basic processing time are sampled from 10 to 50. Worker skill levels follow a discrete uniform distribution from 1 to 3. Skill requirements for levels 1, 2, and 3 are applied to 30, 50, and 20 percent of operations. This hierarchical allocation ensures high constraint tightness and diverse scenarios.

B. Parameter Calibration

All experiments were conducted on an Intel Core i5-12500H CPU @ 2.50 GHz with 16 GB RAM using Python 3.8. The key parameters of MABDQN were calibrated using the Taguchi DOE method. Five parameters affecting the exploration-exploitation balance, learning efficiency, and convergence behavior were selected, each with four levels, population size $PS \in \{100, 150, 200, 250\}$, BDQN learning rate $\alpha \in \{0.0001, 0.0002, 0.0005, 0.001\}$, stagnation threshold $s \in \{0.15, 0.20, 0.25, 0.30\}$, exploration decay $\epsilon \in \{0.99, 0.995, 0.999, 0.9995\}$, and elite ratio $N \in \{0.02, 0.05, 0.08, 0.11\}$. An $L_{16}(4^5)$ orthogonal array generated 16 parameter combinations, each evaluated 20 times on all instances using the relative percentage increase (RPI). Analysis of the main effects plots in Fig. 7 identified the optimal configuration as $PS = 200$, $\alpha = 0.0002$, $s = 0.25$,

$\epsilon = 0.995$, and $N = 0.05$, which was adopted for all subsequent experiments. The constants in the stage adaptive reward function were determined through preliminary empirical tuning to balance feedback sensitivity, with $k_1 = 0.1$, $k_2 = 0.9$, $k_+ = 2.0$, $\alpha_+ = 10$, and $\alpha_- = 5$.

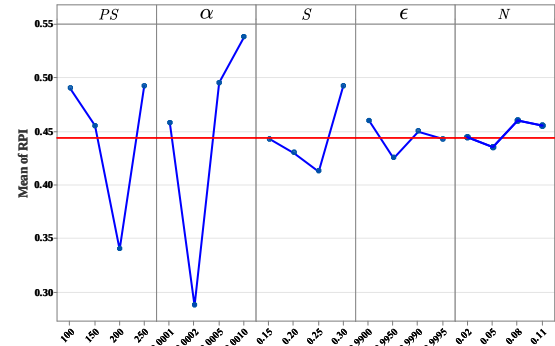
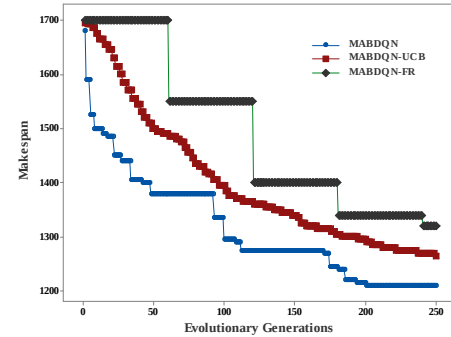


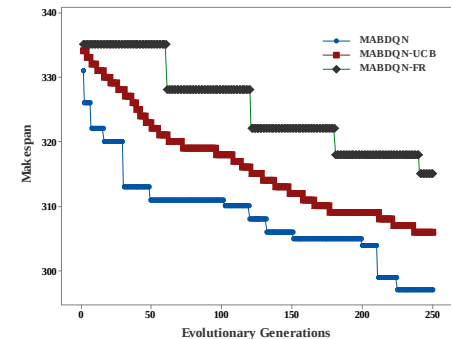
Fig. 7. Main effects plots of factors on the relative percentage increase.

C. Ablation Experiments

Ablation experiments are conducted to evaluate the contribution of each core component in MABDQN. Three variants were considered: random initialization (MABDQN-SI), random operator selection (MABDQN-FR), and the omission of the adaptive restart strategy (MABDQN-NR). Additionally, to verify the state aware learning mechanism, a classical UCB strategy (MABDQN-UCB) is introduced for comparison.



(a) DMFJS10



(b) DMK07

Fig. 8. Convergence curves of MABDQN, MABDQN-UCB and MABDQN-FR on (a) DMFJS10 and (b) DMK07 instances.

Fig. 8 presents the convergence curves. MABDQN converges faster and achieves better solutions than MABDQN-FR

and MABDQN-UCB, indicating that the proposed learning-assisted operator orchestration improves both search efficiency and solution quality. UCB assigns operator credit in a context agnostic manner, where the reward of an operator is averaged over different search phases and resource states. Such a mechanism is less suitable for the SWC-FJSP, in which operator effectiveness is highly non-stationary due to the tight coupling among operation sequencing, machine assignment, and worker assignment. Recombination operators are more beneficial when diverse scheduling structures still exist, whereas resource reallocation operators become more valuable when machine or worker bottlenecks dominate the critical-path. Strong perturbation is useful under stagnation, but may destroy promising structures when the population is still improving. MABDQN alleviates this limitation by conditioning operator decisions on both evolutionary and resource states. The population diversity, fitness trend, improvement rate, convergence phase, and stagnation counter help the agent judge whether the search requires broader exploration, local exploitation, or stronger perturbation, while machine and worker utilization reveal possible resource imbalance. These state signals allow the BDQN policy to estimate operator usefulness under the current search condition, rather than relying on a single global reward record for each operator.

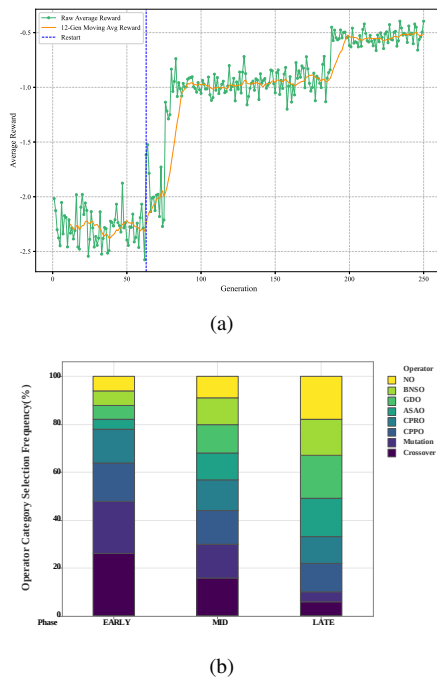


Fig. 9. Performance of the RL agent: (a) average reward curve and (b) operator selection dynamics across evolutionary phases.

The learning behavior in Fig. 9 further supports this interpretation. The average reward curve in Fig. 9(a) gradually increases and becomes more stable, showing that the agent progressively learns actions that produce higher search returns. The operator selection dynamics in Fig. 9(b) indicate that the learned policy does not follow a fixed operator preference throughout the search. In the early stage, the policy assigns more trials to the CR category, implemented by strategies such as OSJC and MSS, to preserve and recombine useful

operation resource structures. As the search proceeds and resource bottlenecks become more evident, the policy shifts toward resource oriented NS operators such as BNSO and CPRO, which can rebalance overloaded machines or workers and adjust critical resource assignments. In the late stage, when the improvement rate becomes smaller and local stagnation is more likely, CPPO and mutation based perturbation are used more frequently to refine critical-path operations or escape local optima. This transition from structural recombination, to bottleneck oriented resource adjustment, and then to critical-path refinement reflects a state conditioned search pattern that is difficult to achieve with stateless reward averaging.

TABLE IV
AVERAGE MAKESPAN AND STANDARD DEVIATION FOR MABDQN AND ITS FOUR VARIANTS.

Ins.	MABDQN-SI	MABDQN-FR	MABDQN-NR	MABDQN-UCB	MABDQN
DMFJS01	470(16) -	497(35) -	462(13) =	461(12) =	459(12)
DMFJS02	376(12) =	424(30) -	379(11) -	375(10) =	374(10)
DMFJS03	445(15) -	490(33) -	435(14) =	434(13) =	431(13)
DMFJS04	541(18) -	680(40) -	537(17) -	531(16) -	523(16)
DMFJS05	550(17) -	573(25) -	536(19) -	532(17) -	526(16)
DMFJS06	614(20) =	794(45) -	617(19) -	613(18) =	611(18)
DMFJS07	895(29) -	925(58) -	877(27) -	873(26) -	865(25)
DMFJS08	915(31) -	940(62) -	878(29) =	881(28) -	877(28)
DMFJS09	1028(35) =	1131(70) -	1036(34) -	1030(33) -	1024(33)
DMFJS10	1259(41) -	1320(450) -	1231(39) -	1224(38) -	1209(38)
DMK01	104(4) -	117(8) -	94(3) -	92(3) =	90(3)
DMK02	91(3) -	94(6) -	98(5) -	89(3) =	88(3)
DMK03	288(9) -	308(10) -	279(18) -	282(10) -	276(8)
DMK04	155(6) -	180(12) -	149(5) -	143(5) -	138(5)
DMK05	364(12) -	383(21) -	359(12) -	355(11) -	349(11)
DMK06	188(7) -	208(15) -	175(6) -	168(6) -	160(5)
DMK07	309(10) -	315(19) -	303(10) -	301(9) -	297(9)
DMK08	675(23) -	841(50) -	665(22) -	658(21) -	647(20)
DMK09	632(22) -	787(48) -	621(20) -	615(19) -	605(18)
DMK10	442(14) -	480(24) -	433(13) -	429(14) -	423(15)
DMW01	7695(240) -	8064(380) -	7540(230) -	7460(215) -	7357(210)
DMW02	10148(300) -	10619(450) -	9950(290) -	9830(280) -	9707(270)
DMW03	6735(200) -	7376(350) -	6605(190) -	6535(180) -	6443(175)
DMW04	5576(165) -	5933(300) -	5466(158) -	5410(150) -	5333(145)
DMW05	4791(150) -	4961(280) -	4702(145) -	4655(135) -	4585(130)
DMW06	6415(190) -	6977(340) -	6288(180) -	6215(172) -	6136(168)
DMW07	5735(175) -	6131(320) -	5622(165) -	5560(155) -	5485(150)
DMW08	7449(225) -	8261(370) -	7301(215) -	7220(205) -	7124(198)
DMW09	8002(250) -	8259(400) -	7844(240) -	7760(230) -	7653(220)
DMW10	8819(270) -	8967(430) -	8645(260) -	8550(250) -	8436(245)
+/-	0/3/27	0/0/30	0/3/27	0/6/24	-

Average makespan and standard deviation are summarized in Table IV, where symbols +, -, and = indicate that the corresponding variant performs significantly better, worse, or comparably with MABDQN, respectively. The full MABDQN achieves the best overall performance across all instances. The inferior performance of MABDQN-FR indicates that blind operator selection cannot effectively handle the tight constraints caused by worker skills and machine flexibility. MABDQN-UCB performs better than random selection, but it remains inferior to MABDQN in most cases, suggesting that adaptive selection based only on historical rewards is still not sufficient for this highly constrained problem. By jointly considering population evolution and machine-worker utilization, MABDQN can better preserve promising scheduling structures, reallocate overloaded resources when bottlenecks emerge, and introduce stronger perturbations when stagnation accumulates. This state aware and intensity aware orchestration explains its better convergence behavior and solution quality. Furthermore,

the performance drops of MABDQN-SI and MABDQN-NR confirm that problem aware initialization and adaptive restart mechanisms are also important for avoiding premature convergence in the fragmented search space of the SWC-FJSP.

TABLE V
SENSITIVITY ANALYSIS OF DIFFERENT DQN STRUCTURES IN TERMS OF AVERAGE RELATIVE PERCENTAGE INCREASE (%)

Variant	Action	Network (S/B/O)	Small	Medium	Large	Average
Flat-DQN	24 joint	128-128/64/24	2.65	1.58	3.24	2.49
BDQN-S	8 category/3 intensity	64-64/32/8, 3	1.74	1.88	2.15	1.92
BDQN-M	8 category/3 intensity	128-128/64/8, 3	2.36	1.35	0.08	1.26
BDQN-L	8 category/3 intensity	256-256/128/8, 3	2.86	0.94	0.82	1.54
BDQN-D	8 category/3 intensity	128-128-128/64/8, 3	1.66	1.22	0.95	1.28

To examine the effect of the DQN architecture, five variants were tested under the same state, reward, replay buffer, and training settings. Flat-DQN removes the branching design by using one joint action head, BDQN-S, BDQN-M, and BDQN-L vary the network width, and BDQN-D increases the shared-network depth. BDQN-M is the architecture adopted in MABDQN. In Table V, S/B/O denotes the shared layers, branch or head layer, and output heads. The average relative percentage increase (ARPI) is computed for each instance relative to the best mean makespan among all tested structures and then averaged by instance group. Flat-DQN gives the worst overall ARPI and the largest degradation on large instances, indicating that coupling operator category and intensity into one action head weakens action-value learning. Among the branched variants, BDQN-S shows limited improvement, while BDQN-L and BDQN-D improve only some instance groups. BDQN-M obtains the lowest overall ARPI and the best large-scale ARPI, supporting its use as the empirical trade-off among solution quality, scalability, and online learning stability.

TABLE VI
COMPARISON OF AVERAGE CPU TIME AND ONLINE RL TRAINING OVERHEAD BY SCALE

Scale	MABDQN-FR(s)	MABDQN(s)	Overhead(%)
Small	20.49	24.18	18.01%
Medium	90.82	104.44	14.99%
Large	1278.50	1393.56	9.00%

Table VI reports the average CPU time and online RL training overhead by scale. The reported training overhead includes transition storage, mini-batch BDQN updates, and target network synchronization, and no offline pretraining is used. The overhead is 18.01%, 14.99%, and 9.00% on small, medium, and large-scale instances, respectively. Although online RL training introduces extra computation, its relative cost decreases as the instance scale increases. This is because schedule decoding, offspring evaluation, and evolutionary operators become more time consuming on larger instances, while the BDQN network structure and mini-batch size remain fixed. Therefore, the RL training cost is gradually amortized by the scheduling search cost, indicating that the additional training burden is moderate compared with the improvement in solution quality.

D. Comparison with Other Algorithms

To evaluate the effectiveness of MABDQN, seven representative algorithms were selected for comparison on all instances, including SMOSA [40], IIG [41], HQPSO [42], HGTS [43], FLMA [44], APHMA [45], and DRLAGA [46]. SMOSA, IIG, HQPSO, and HGTS are problem relevant baselines that cover widely used search paradigms for FJSP and related scheduling problems, including simulated annealing, iterated greedy search, swarm based optimization, and tabu search based hybrid search. FLMA and APHMA are recent state-of-the-art MAs for advanced FJSP variants, providing strong competitors from the evolutionary optimization perspective. DRLAGA is further included as a deep reinforcement learning-assisted evolutionary algorithm, which enables a direct comparison with learning-assisted search mechanisms.

To ensure a controlled comparison, all baselines were evaluated under a common SWC-FJSP protocol while preserving their search cores. The protocol uses the same operation, machine, and worker representation, skill feasible resource sets, skill adjusted processing times, active decoding rule, and makespan objective. For methods without worker assignment, a worker segment was added only to fit the SWC-FJSP decision structure. The original mechanisms, including annealing, reconstruction, particle update, tabu search, feedback learning, affinity propagation based population management, and genetic search assisted by deep reinforcement learning, were retained. Parameters followed the original studies when applicable, otherwise they were tuned in preliminary tests and fixed for all instances. Each algorithm was run 20 times under the same environment and stopped at $PS \times G_{\max} = 50,000$ function evaluations. Thus, the results reflect search performance under the same modeling, decoding, evaluation, and budget settings.

The average makespan and standard deviation on all instances are reported in Table VII. MABDQN achieves the best overall performance among the eight algorithms. Although several baselines obtain competitive results on small instances, where the search space is relatively limited, the advantage of MABDQN becomes clear as the problem scale increases. It achieves the best results on all medium and large instances, with smaller or comparable standard deviations in most cases. Its superiority over FLMA and APHMA further indicates stronger scalability and robustness under coupled operation, machine, worker, and skill constraints. This performance gain can be attributed to the BDQN guided operator orchestration and adaptive restart, which help the search adjust its exploration and exploitation behavior.

The ARPI distributions in Fig. 10 provide a scale independent comparison of all algorithms. MABDQN is concentrated near zero and shows the lowest median ARPI with the narrowest interquartile range, indicating stable performance across different instance scales. In contrast, most baselines exhibit higher medians and longer upper tails, showing larger performance loss and stronger instance sensitivity. Pairwise one way ANOVA tests between MABDQN and the seven baselines further support this observation. The differences are significant at the 0.05 level for all baselines except DRLAGA,

TABLE VII
AVERAGE MAKESPAN AND STANDARD DEVIATION FOR MABDQN AND OTHER COMPARED ALGORITHMS.

Ins.	SMOSA [40]	IIG [41]	HQPSO [42]	HGTS [43]	FLMA [44]	APHMA [45]	DRLAGA [46]	MABDQN
DMFJS01	437(15) +	450(18) -	475(20) -	462(19) -	430(11) +	449(23) +	474(21) -	459(12)
DMFJS02	378(12) =	418(16) -	416(15) -	387(13) -	382(12) -	392(15) -	385(12) -	374(10)
DMFJS03	486(18) -	458(17) -	473(20) -	433(15) =	455(30) -	460(26) -	453(16) -	431(13)
DMFJS04	568(20) -	580(22) -	581(23) -	528(18) =	544(24) -	556(40) -	545(19) -	523(16)
DMFJS05	535(19) -	585(24) -	595(25) -	542(20) -	548(27) -	526(22) =	513(18) +	526(16)
DMFJS06	675(25) -	680(26) -	697(28) -	610(20) =	655(26) -	629(22) -	608(22) =	611(18)
DMFJS07	880(30) -	875(29) -	910(35) -	845(28) +	891(15) -	882(21) -	859(31) -	865(25)
DMFJS08	974(35) -	875(30) =	940(38) -	890(32) -	941(36) -	966(49) -	910(34) -	877(28)
DMFJS09	1139(40) -	1120(38) -	1155(45) -	1116(39) -	1150(34) -	1157(20) -	1120(42) -	1024(33)
DMFJS10	1316(45) -	1325(48) -	1350(55) -	1278(44) -	1274(27) -	1391(79) -	1265(46) -	1209(38)
DMK01	133(5) -	109(4) -	115(6) -	92(4) =	99(2) -	88(2) +	93(4) =	90(3)
DMK02	135(6) -	128(5) -	118(7) -	103(5) -	98(4) -	103(6) -	98(5) -	88(3)
DMK03	479(15) -	352(12) -	388(18) -	337(13) -	325(6) -	356(10) -	317(12) -	276(8)
DMK04	216(9) -	180(8) -	195(10) -	139(7) =	143(4) -	152(8) -	144(8) -	138(5)
DMK05	452(18) -	395(16) -	410(19) -	356(15) -	370(12) -	374(6) -	367(16) -	349(11)
DMK06	343(12) -	215(9) -	240(11) -	184(8) -	201(7) -	213(11) -	189(9) -	160(5)
DMK07	450(18) -	385(15) -	405(19) -	341(14) -	320(6) -	322(8) -	306(12) -	297(9)
DMK08	1007(28) -	850(25) -	880(30) -	728(24) -	725(20) -	751(16) -	693(23) -	647(20)
DMK09	898(26) -	795(24) -	812(28) -	705(22) -	671(17) -	702(28) -	660(21) -	605(18)
DMK10	664(22) -	590(20) -	615(24) -	524(19) -	511(19) -	529(17) -	492(18) -	423(15)
DMW01	9587(291) -	9852(318) -	10550(324) -	11202(352) -	9637(257) -	9912(312) -	8723(280) -	7357(210)
DMW02	12730(384) -	13100(396) -	13526(424) -	11473(360) -	10817(233) -	11475(489) -	11024(355) -	9707(270)
DMW03	8972(283) -	9530(274) -	9836(310) -	9135(290) -	9154(210) -	9960(187) -	8016(262) -	6443(175)
DMW04	7365(234) -	7900(256) -	8150(267) -	7345(235) -	7239(166) -	8233(203) -	6883(220) -	5333(145)
DMW05	7413(240) -	7800(263) -	8000(279) -	6356(210) -	7743(123) -	7706(256) -	5871(195) -	4585(130)
DMW06	9685(305) -	10100(320) -	10400(344) -	8058(273) -	9813(329) -	9479(477) -	7680(257) -	6136(168)
DMW07	8721(290) -	9200(313) -	9500(334) -	8624(295) -	8824(249) -	9815(129) -	7320(243) -	5485(150)
DMW08	11699(366) -	12200(385) -	12600(405) -	9308(310) -	9227(307) -	10158(202) -	9013(308) -	7124(198)
DMW09	13171(410) -	13800(430) -	14200(450) -	10096(346) -	10012(177) -	10981(429) -	9945(330) -	7653(220)
DMW10	15198(482) -	15800(518) -	16300(527) -	13271(440) -	14261(228) -	13860(322) -	12520(425) -	8436(245)
+/-	1/1/28	0/1/29	0/0/30	1/5/24	1/0/29	2/1/27	1/2/27	-

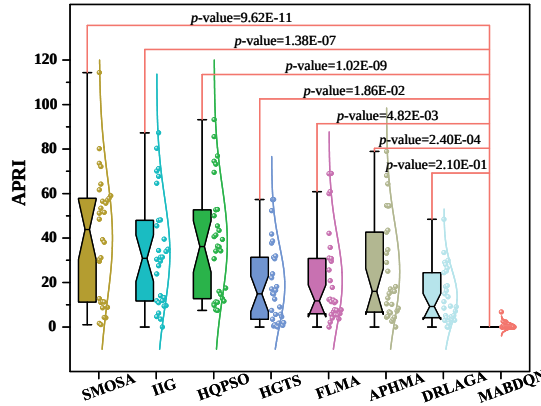


Fig. 10. Box-plot comparison of algorithms in terms of ARPI.

whose learning assisted evolutionary search yields the closest statistical behavior. Even in this case, MABDQN still shows a more compact ARPI distribution, confirming its robustness and consistency.

The Friedman ranking based on average makespan is given in Table VIII. MABDQN obtains the best average rank of 1.38. The Friedman test yields a p -value of $1.0387\text{E-}25$, indicating significant performance differences among the eight algorithms. Together with the instance level results and ARPI distributions, these rankings show that MABDQN provides the most consistent search performance for the SWC-FJSP, especially under large-scale operation, machine, worker, and skill coupling.

TABLE VIII
AVERAGE FRIEDMAN RANKINGS OF MABDQN AND COMPARED ALGORITHMS ON AVERAGE MAKESPAN.

Algorithm	Rank	p -value
SMOSA	5.97	$1.0387\text{E-}25$
IIG	5.92	
HQPSO	7.27	
HGTS	3.58	
FLMA	3.98	
APHMA	5.27	
DRLAGA	2.63	
MABDQN	1.38	

VI. CONCLUSION

This article proposes an MABDQN framework for the SWC-FJSP, featuring adaptive hybrid initialization, BDQN-driven operator scheduling, and an adaptive restart strategy. The framework jointly addresses operation sequencing, machine allocation, and worker assignment under skill-based feasibility and skill-dependent processing efficiency. Extensive experiments and ablation studies demonstrate that MABDQN significantly outperforms state-of-the-art algorithms in solution quality and stability, with BDQN effectively balancing exploration and exploitation during evolutionary search. The results also confirm that adaptive initialization and restart strategies improve robustness in the constrained solution space. Future work will extend MABDQN to dynamic scenarios, multi-objective optimization, and complex worker behaviors such as fatigue, learning, and forgetting effects. Exploring

advanced DRL architectures in distributed multi-factory environments also remains a promising direction.

REFERENCES

- [1] J. Li, X. Li, L. Gao, and Q. Liu, "A flexible job shop scheduling problem considering on-site machining fixtures: A case study from customized manufacturing enterprise," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 8415–8426, Oct. 2024.
- [2] R. Li, W. Gong, C. Lu, and L. Wang, "A learning-based memetic algorithm for energy-efficient flexible job-shop scheduling with type-2 fuzzy processing time," *IEEE Trans. Evol. Comput.*, vol. 27, no. 3, pp. 610–620, May 2022.
- [3] X. He, Q. Pan, L. Gao, J. S. Neufeld, and J. N. Gupta, "Historical information based iterated greedy algorithm for distributed flowshop group scheduling problem with sequence-dependent setup times," *Omega*, vol. 123, Feb. 2024, Art. no. 102997.
- [4] Q. Luo, Q. Deng, G. Xie, and G. Gong, "A pareto-based two-stage evolutionary algorithm for flexible job shop scheduling problem with worker cooperation flexibility," *Robot. Comput.-Integr. Manuf.*, vol. 82, Aug. 2023, Art. no. 102534.
- [5] H. Song, J. Li, Z. Du, X. Yu, Y. Xu, Z. Zheng, and J. Li, "A Q-learning driven multi-objective evolutionary algorithm for worker fatigue dual-resource-constrained distributed hybrid flow shop," *Comput. Oper. Res.*, vol. 175, Mar. 2025, Art. no. 106919.
- [6] J. Li, X. Li, L. Gao, C. Wang, and H. Chen, "A multitasking workforce-constrained flexible job shop scheduling problem: An application from a real-world workshop," *J. Manuf. Syst.*, vol. 83, pp. 196–215, Dec. 2025.
- [7] J. Li, Y. Han, K. Gao, X. Xiao, and P. Duan, "Bi-population balancing multi-objective algorithm for fuzzy flexible job shop with energy and transportation," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, pp. 4686–4702, Aug. 2023.
- [8] X. Xiangqian, Y. Kewei, D. Yajie, Z. Zhexuan, C. Ziyi, and T. Yuejin, "High-end equipment development task decomposition and scheme selection method," *J. Syst. Eng. Electron.*, vol. 32, no. 1, pp. 118–135, Feb. 2021.
- [9] H. Ma, X. Huang, Z. Hu, Y. Chen, D. Qian, J. Deng, and L. Hua, "Multi-objective production scheduling optimization and management control system of complex aerospace components: a review," *Int. J. Adv. Manuf. Technol.*, vol. 127, no. 11, pp. 4973–4993, Jul. 2023.
- [10] G. Teng, "An improved genetic algorithm for dual-resource constrained flexible job shop scheduling problem with tool-switching dependent setup time," *Expert Syst. Appl.*, vol. 281, Jul. 2025, Art. no. 127496.
- [11] H. Bao, Q.-K. Pan, C.-M. Chew, L. Wang, and L. Gao, "Energy-efficient distributed heterogeneous hybrid flow-shop scheduling using graph neural network and deep reinforcement learning," *IEEE Trans. Autom. Sci. Eng.*, 2025.
- [12] C. Zhang, M. Rong, S. Yang, Q.-K. Pan, and C. Peng, "Dynamic multi-objective optimization for integrated coal mine energy systems with streaming constraints," *IEEE Trans. Ind. Inform.*, vol. 22, no. 4, pp. 3319–3329, Feb. 2026.
- [13] K. Han and W. Gong, "Memetic algorithm based on non-dominated levels for flexible job shop scheduling problem with learn-forgetting effect and worker cooperation," *Comput. Ind. Eng.*, vol. 200, Feb. 2025, Art. no. 110845.
- [14] K. Jarosz, Y. Chen, and R. Liu, "Investigating the differences in human behavior between conventional machining and CNC machining for future workforce development: A case study," *J. Manuf. Process.*, 2023.
- [15] Ö. Yildirim, A. A. Kiss, and E. Y. Kenig, "Dividing wall columns in chemical process industry: A review on current activities," *Sep. Purif. Technol.*, vol. 80, no. 3, pp. 403–417, Aug. 2011.
- [16] M. Al Amin, R. Baldacci, and V. Kayvanfar, "A comprehensive review on operating room scheduling and optimization," *Oper. Res.*, vol. 25, no. 1, Nov. 2024, Art. no. 3.
- [17] C. Li, L. Meng, S. Ullah, P. Duan, B. Zhang, and H. Sang, "Novel hybrid algorithm of cooperative evolutionary algorithm and constraint programming for dual resource constrained flexible job shop scheduling problems," *Complex Syst. Model. Simul.*, vol. 5, no. 3, pp. 236–251, Apr. 2025.
- [18] X. Li, L. Gao, Q. Pan, L. Wan, and K. Chao, "An effective hybrid genetic algorithm and variable neighborhood search for integrated process planning and scheduling in a packaging machine workshop," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 49, no. 10, pp. 1933–1945, Dec. 2018.
- [19] X. Li and S. Xing, "Dynamic scheduling problem of multi-objective dual resource flexible job shop based on improved genetic algorithm," in *Proc. Chinese Control Conf. (CCC)*. IEEE, Jul. 2024, pp. 2046–2051.
- [20] R. Li, W. Gong, L. Wang, C. Lu, and X. Zhuang, "Surprisingly popular-based adaptive memetic algorithm for energy-efficient distributed flexible job shop scheduling," *IEEE Trans. Cybern.*, vol. 53, no. 12, pp. 8013–8023, Jun. 2023.
- [21] J. Wang and L. Wang, "A cooperative memetic algorithm with learning-based agent for energy-aware distributed hybrid flow-shop scheduling," *IEEE Trans. Evol. Comput.*, vol. 26, no. 3, pp. 461–475, Aug. 2021.
- [22] Y. Wang, Y. Han, Y. Wang, X. Wang, Y. Liu, and K. Gao, "Reinforcement learning-assisted memetic algorithm for sustainability-oriented multiobjective distributed flow shop group scheduling," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 55, no. 4, pp. 2399–2413, Jan. 2025.
- [23] H. Wei, S. Li, H. Jiang, J. Hu, and J. Hu, "Hybrid genetic simulated annealing algorithm for improved flow shop scheduling with makespan criterion," *Appl. Sci.*, vol. 8, no. 12, Dec. 2018, Art. no. 2621.
- [24] X. Gong, Q. Deng, G. Gong, W. Liu, and Q. Ren, "A memetic algorithm for multi-objective flexible job-shop problem with worker flexibility," *Int. J. Prod. Res.*, vol. 56, no. 7, pp. 2506–2522, Sep. 2017.
- [25] Z. Zhang, L. Gao, X. Li, C. Zhang, and Q. Liu, "Solving flexible job shop scheduling problem with worker flexibility and outsourcing service time windows using an adaptive dual-population memetic algorithm," *Int. J. Prod. Res.*, vol. 64, no. 8, pp. 3210–3243, Apr. 2026.
- [26] J. Zhang, Z. Zhan, Y. Lin, N. Chen, Y. Gong, J. Zhong, H. S. Chung, Y. Li, and Y. Shi, "Evolutionary computation meets machine learning: a survey," *IEEE Comput. Intell. Mag.*, vol. 6, no. 4, pp. 68–75, Nov. 2011.
- [27] Z. Zhan, J. Li, S. Kwong, and J. Zhang, "Learning-aided evolution for optimization," *IEEE Trans. Evol. Comput.*, vol. 27, no. 6, pp. 1794–1808, Dec. 2023.
- [28] Y. Jiang, Z. Zhan, K. C. Tan, and J. Zhang, "Knowledge learning for evolutionary computation," *IEEE Trans. Evol. Comput.*, vol. 29, no. 1, pp. 16–30, Feb. 2025.
- [29] Y. Jin, "Surrogate-assisted evolutionary computation: recent advances and future challenges," *Swarm Evol. Comput.*, vol. 1, no. 2, pp. 61–70, Jun. 2011.
- [30] S. Liu, Q. Lin, J. Li, and K. C. Tan, "A survey on learnable evolutionary algorithms for scalable multiobjective optimization," *IEEE Trans. Evol. Comput.*, vol. 27, no. 6, pp. 1941–1961, Dec. 2023.
- [31] A. Gupta, Y.-S. Ong, and L. Feng, "Insights on transfer optimization: because experience is the best teacher," *IEEE Trans. Emerg. Top. Comput. Intell.*, vol. 2, no. 1, pp. 51–64, Feb. 2018.
- [32] B. Wu, T. Chen, W. Ni, and X. Wang, "Multi-agent multi-armed bandit learning for online management of edge-assisted computing," *IEEE Trans. Commun.*, vol. 69, no. 12, pp. 8188–8199, Aug. 2021.
- [33] S. Zhang, Z. Ren, C. Li, and J. Xuan, "A perturbation adaptive pursuit strategy based hyper-heuristic for multi-objective optimization problems," *Swarm Evol. Comput.*, vol. 54, May 2020, Art. no. 100647.
- [34] D. E. Goldberg, "Probability matching, the magnitude of reinforcement, and classifier system bidding," *Mach. Learn.*, vol. 5, no. 4, pp. 407–425, Oct. 1990.
- [35] A. Tavakoli, F. Pardo, and P. Kormushev, "Action branching architectures for deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1. AAAI Press, 2018, pp. 4131–4138.
- [36] C.-L. Liu and T.-H. Huang, "Dynamic job-shop scheduling problems using graph neural network and deep reinforcement learning," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 53, no. 11, pp. 6836–6848, Jul. 2023.
- [37] L. Zheng, X. Chen, C. Zhuang, J. Liu, Y. Zhang, and L. Lai, "Dynamic scheduling for flexible job-shop with reconfigurable manufacturing cells considering dynamic job arrivals based on deep reinforcement learning," *Int. J. Prod. Res.*, vol. 63, no. 20, pp. 7427–7459, Apr. 2025.
- [38] P. Fattahi, M. Saidi Mehrabad, and F. Jolai, "Mathematical modeling and heuristic approaches to flexible job shop scheduling problems," *J. Intell. Manuf.*, vol. 18, no. 3, pp. 331–342, Jul. 2007.
- [39] P. Brandimarte, "Routing and scheduling in a flexible job shop by tabu search," *Ann. Oper. Res.*, vol. 41, no. 3, pp. 157–183, Sep. 1993.
- [40] F. M. Defersha, D. Obimuyiwa, and A. D. Yimer, "Mathematical model and simulated annealing algorithm for setup operator constrained flexible job shop scheduling problem," *Comput. Ind. Eng.*, vol. 171, Sep. 2022, Art. no. 108487.
- [41] W. Li, X. Chen, J. Li, H. Sang, Y. Han, and S. Du, "An improved iterated greedy algorithm for distributed robotic flowshop scheduling with order constraints," *Comput. Ind. Eng.*, vol. 164, Feb. 2022, Art. no. 107907.
- [42] Y. Xu, M. Zhang, M. Yang, and D. Wang, "Hybrid quantum particle swarm optimization and variable neighborhood search for flexible job-shop scheduling problem," *J. Manuf. Syst.*, vol. 73, pp. 334–348, Apr. 2024.

- [43] J. Xie, X. Li, L. Gao, and L. Gui, "A hybrid genetic tabu search algorithm for distributed flexible job shop scheduling problems," *J. Manuf. Syst.*, vol. 71, pp. 82–94, Dec. 2023.
- [44] J. Wang, H. Han, and L. Wang, "A feedback learning-based memetic algorithm for energy-aware distributed flexible job-shop scheduling with transportation constraints," *IEEE Trans. Evol. Comput.*, vol. 29, no. 4, pp. 1085–1099, Aug. 2025.
- [45] C. Luo, X. Li, W. Gong, and L. Gao, "Affinity propagation hierarchical memetic algorithm for multimodal multiobjective flexible job shop scheduling with variable speed," *IEEE Trans. Evol. Comput.*, vol. 29, no. 6, pp. 2729–2741, Dec. 2025.
- [46] J. Ma, W. Gao, and W. Tong, "A deep reinforcement learning assisted adaptive genetic algorithm for flexible job shop scheduling," *Eng. Appl. Artif. Intell.*, vol. 149, Jun. 2025, Art. no. 110447.



Chi Zhang is currently pursuing the Ph.D. degree with the School of Mechatronic Engineering and Automation, Shanghai University, Shanghai, China.

His research interests include intelligent optimization, reinforcement learning, federated learning, and intelligent scheduling.



Quanke Pan (Member, IEEE) received the B.Sc. degree and the Ph.D. degree from Nanjing university of Aeronautics and Astronautics, Nanjing, China, in 1993 and 2003, respectively.

From 2003 to 2011, he was with School of Computer Science Department, Liaocheng University, where he became a Full Professor in 2006. From 2011 to 2014, he was with State Key Laboratory of Synthetical Automation for Process Industries (Northeastern University), Shenyang, China. From 2014 to 2015, he was with State Key Laboratory

of Digital Manufacturing and Equipment Technology (Huazhong University of Science and Technology). He has been with School of Mechatronic Engineering and Automation, Shanghai University since 2015. His current research interests include intelligent optimization and scheduling algorithms. He has authored two academic books and more than 300 refereed papers. He acts as the Editorial Board Member for several journals, including *Operations Research Perspectives* and *Swarm and Evolutionary Computation*.



Ling Wang (Senior Member, IEEE) received the B.Sc. degree in automation and the Ph.D. degree in control theory and control engineering from the Tsinghua University, Beijing, China, in 1995 and 1999, respectively.

Since 1999, he has been with the Department of Automation, Tsinghua University, where he became a Full Professor in 2008. He has authored five academic books and more than 300 refereed papers. His research interests include intelligent optimization and production scheduling.

Prof. Wang is a recipient of the National Natural Science Fund for Distinguished Young Scholars of China, the National Natural science Award (Second Place) in 2014, the Science and Technology Award of Beijing City in 2008, the Natural Science Award (First Place in 2003 and Second Place in 2007) nominated by the Ministry of Education of China. He is currently the Editor-in-Chief of *International Journal of Automation and Control*, *Swarm and Evolutionary Computation*, *Expert Systems with Applications*, and an Associate Editor of *IEEE Transactions on Evolutionary Computation*.



Zhonghua Miao received the Ph.D. degree in mechatronic engineering from Shanghai Jiao Tong University (SJTU), Shanghai, China, in 2010.

He is currently a Professor and the Vice Dean of the School of Mechatronic Engineering and Automation, Shanghai University, Shanghai. He is also a Distinguished Professor with the Chang Jiang Scholars Program of China. His current research interests include intelligent robotics, specifically focusing on visual perception, trajectory planning, decision-making control, and multi-robot coordination.



Weimin Li received the Ph.D. degree in control theory and control engineering from the School of Information Engineering and Science, Donghua University, Shanghai, China, in 2008. In the same year, he joined the faculty of the School of Computer Engineering and Science, Shanghai University, Shanghai, China, where he is currently a Professor.

He was a JSPS Research Fellow with the Department of Human Informatics and Cognitive Sciences, Waseda University, Tokyo, Japan, from 2012 to 2013. Moreover, he was a Visiting Scholar with

the Department of Computer Science, University of California at Santa Barbara, Santa Barbara, CA, USA, from 2015 to 2016, supported by the China Scholarship Council. He is currently a Professor with the School of Computer Engineering and Science, Shanghai University, Shanghai, China. He has extensively researched computer science, service computing, business process management, and database technology. His research interests include social computing, data mining and analytics, group behavior modeling and simulating, and service recommendations.